OIDC e federação com a nuvem

GH-200 | Complemento 2026 | Resumo de estudo para Cledson

1. O que resolve

OpenID Connect (OIDC) permite que um job comprove sua identidade ao provedor de nuvem e obtenha credenciais temporárias. A federação estabelece confiança entre o emissor GitHub e a identidade ou role configurada no provedor. Isso evita manter uma chave de acesso de longa duração da nuvem como secret no GitHub. [1]

2. Fluxo que você precisa entender

1. O administrador configura a confiança no provedor.
2. Uma action de autenticação solicita ao GitHub um JWT OIDC.
3. O provedor valida assinatura, validade e claims.
4. Se as condições coincidirem, emite credenciais temporárias.
5. O job acessa somente os recursos autorizados para aquela identidade. [1]

3. Três credenciais diferentes

GITHUB_TOKEN: autenticação para operações no GitHub.
Token OIDC: prova da identidade do job apresentada ao provedor.
Credenciais da nuvem: resultado da troca, usado para acessar os serviços. O JWT do GitHub não é, por si só, uma chave de acesso ao S3 ou ao Azure. [1, 3, 4]

4. A permissão que cai na prova

```
permissions:
  id-token: write # Permite solicitar o JWT OIDC.
  contents: read # Permite leitura para checkout, se necessário.
```

write não significa escrever na nuvem. Essa permissão habilita a solicitação do token; a autorização vem das políticas da nuvem. Pode ser definida no workflow ou somente no job que precisa autenticar. Declarar a permissão também não faz login sozinho: é necessário um step que solicite e troque o token. [3]

Exemplo comentado: AWS

Pré-requisitos: provedor OIDC configurado no IAM, role com política de confiança e permissões mínimas. Issuer: https://token.actions.githubusercontent.com. Na integração padrão AWS, a audience é sts.amazonaws.com. [3]

```
name: Verificar identidade AWS
on:
  workflow_dispatch: # Disparo manual.
jobs:
  autenticar:
    runs-on: ubuntu-latest # OIDC não exige Docker.
    permissions:
      id-token: write # Solicita o token de identidade.
    steps:
      - name: Obter credenciais temporárias
        # SHA usado no exemplo oficial consultado.
        uses: aws-actions/configure-aws-credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          # Substitua pelo ARN da role já configurada.
          role-to-assume: arn:aws:iam::123456789012:role/DeployRole
          aws-region: us-east-1 # Região da operação.
      - name: Confirmar a identidade
        run: aws sts get-caller-identity # Não imprime o token.
```

Não há aws-access-key-id nem aws-secret-access-key: a action solicita o JWT e faz a troca com AWS STS. O ARN identifica a role, mas não é uma senha. Este exemplo não usa checkout porque não lê arquivos do repositório. [3]

Condições de confiança: iss, aud e sub

iss: quem emitiu o token.
aud: destinatário esperado.
sub: identidade/contexto aceito pelo provedor. Restrinja a confiança ao repositório e contexto pretendidos; confiar em qualquer job do GitHub seria amplo demais. [2]

```
# Exemplos do formato anterior de sub:
repo:acme/app:ref:refs/heads/main
repo:acme/app:environment:producao
repo:acme/app:pull_request
```

Com environment, o subject padrão usa o contexto do ambiente. Configure suas regras de proteção e branches de deploy. Não copie uma condição de branch para um job com environment sem conferir o subject efetivo. [2, 3]

Pegadinhas e diagnóstico

Complemento 2026: conferir o subject

A referência atual informa que repositórios criados após 15/07/2026, ou que aderiram ao formato imutável, usam IDs de proprietário e repositório no sub. Exemplo: repo:acme@123/app@456:ref:refs/heads/main. Essa mudança não está disponível no GitHub Enterprise Server. Ajuste a confiança ao formato real do repositório; os exemplos antigos continuam relevantes para repositórios que os utilizam. [2]

Azure: a mesma ideia com outra configuração

Use uma credencial federada na identidade do Microsoft Entra e uma action de login, como azure/login. Client ID, tenant ID e subscription ID identificam a configuração e não substituem a credencial federada. A audience padrão da action é api://AzureADTokenExchange. As permissões no Azure dependem dos papéis atribuídos à identidade. [4]

Como interpretar falhas

Não consegue solicitar JWT: verifique id-token: write e as permissões efetivas.
Token rejeitado na troca: compare issuer, audience, subject e validade com a confiança.
Login passa, deploy é negado: investigue a autorização da identidade na nuvem, não apenas OIDC.
Falha ao mudar branch ou environment: confira se o contexto mudou o subject esperado. [2, 3, 4]

Guarde para a prova

OIDC autentica o job e permite obter credenciais temporárias. Não concede acesso universal, não exige runner Linux e não elimina todos os secrets da aplicação. Não registre o JWT ou credenciais nos logs. Confiança e autorização são configurações separadas. Em reusable workflows, confira as permissões efetivas do chamador e a identidade/contexto avaliado. [1, 2]

Fontes oficiais consultadas em 11/10/2026

[1] GitHub Docs: Conceitos OIDC

[2] GitHub Docs: Referência OIDC e claims

[3] GitHub Docs: Configurar OIDC na AWS

[4] GitHub Docs: Configurar OIDC no Azure